

Real-time Reconfigurable Cache for Embedded Systems

Geng-Cyuan Jheng¹, Dyi-Rong Duh¹, and Cheng-Nan. Lai²

¹Department of Computer Science and Information Engineering
National Chi Nan University, Puli, Nantou Hsien, 54561 Taiwan
{s95321515, drduh}@ncnu.edu.tw

²Department of Information Management
National Kaohsiung Marine University, Kaohsiung City, 81143 Taiwan
cnlai@mail.nkmu.edu.tw

Abstract

Modern embedded systems execute a small set of applications or even a single one repeatedly. Specializing cache configurations to a particular application is well-known to have great benefits on performance and power. To reduce the searching for optimal cache configuration, a most-case optimal cache configuration searching algorithm was proposed which greatly reduces the time and power in searching. However, the fact that the behavior of an application varies from phase to phase has been shown in recent years. Tuning cache configuration to fit a target application in different phases gives a further improvement in power consumption. This work presents a mechanism which determines the optimal configurations in different phases during an execution process. By dividing an execution process into small time intervals and applying corresponding local optimal cache configuration for each interval on L1 instruction cache, this work shows that over 60% energy saving is obtained compared with conventional four-way set-associativity and conventional direct-mapped caches. On average 7.59% power reduction is achieved compared to a benchmark with its respective global optimal cache configuration.

1 Introduction

The research of computer systems is growing and is getting more important in the last three decades. The gap between the processor and the memory has widened when computer systems improve. It has been shown that inserting caches between processors and off-chip memories fills the gap efficiently. Embedded microprocessors have been widely used for the hand held devices due to their high performance, low energy consumption, and low cost. Caches are infixed in embedded microprocessors with the growth of embedded systems. The most important thing for embedded systems like PDA and cellular phone is

how long the device is alive. Segars [8] showed that an embedded microprocessor such as the ARM920T processor dissipates more than 50% of the total power in the cache. With the increasing usage of cache, energy savings in caches have become more critical.

Sherwood et al. [9] showed that programs may have different run-time behaviors in different portions of an execution as a series of phases. A phase is defined as an interval of execution during which a measured program metric is relatively stable. The scheme proposed by Sherwood et al. captures phases that account for over 80% of execution using less than 500 bytes of on-chip memory [9].

Some strategies augmented small caches between the processor and lower-level cache to reduce the energy consumption, such as the filter cache introduced by Kin et al. [6] and the hotspot cache introduced by Yang et al. [13]. Others improved the cache itself to reduce the energy consumption. Many researches on caches with tunable parameters to reduce power consumption have been introduced in recently years. Zhang et al. [14] introduced a highly configurable cache architecture which dynamically configures the associativity, size and line size with certain restrict combinations. Banerjee et al. [1] combined both program phase detection and cache way reconfigurable to achieve 32% power reduction on memory access with less than 2% performance degradation.

A single-pass multi-cache evaluation has been proposed and implemented in hardware by Gordon-Ross et al. [4] which speeds up 7.7 times in tuning time. Even so, the hardware overhead is so tremendous which occupies up to 12% area of ARM920T. Lin and Duh proposed an on-line reconfigurable cache which greatly reduces the search time and the cache tuning can proceed during the execution [7]. The study similar to Lin et al.'s work is proposed by Chen et al [3]. The algorithm is almost the same but Lin goes further in statistics. This work is based on Lin et al.'s work and enhances the scheme to real-time tuning.

The remainder of this paper is organized as follows. Section 2 first introduces the architecture of reconfigurable cache. The energy model is then presented. Section 3 presents how to implement a real-time reconfigurable cache system. Section 4 demonstrates experimental methodology and results. Conclusions are finally drawn in Section 5.

2 Reconfigurable Architecture

2.1 Parameters

Instead of adopting conventional cache parameters such as cache size, cache block size, and way associativity, the proposed reconfigurable cache architecture includes three parameters: the number of blocks in a set, cache block size, and way associativity. The three parameters are *nblks*, *bsize*, and *assoc* for short in this paper. Cache size is composed of *nblks*, *bsize* and *assoc*. The parameter *nblks* is chosen to replace cache size because of the “simple and dump” principle. Considering hardware implementation, it is under a complicated circumstance for the conventional cache parameters. Changing cache size affects *nblks* simultaneously and so are the other two parameters. As cache size is not a parameter in this work, tuning one of the three parameters just simply doubles or halves the cache size and does not affect other parameters.

The bounds of configuration search space are decided by extending Lin et al.’s work [7]. The upper bound of *nblks* is 512 and the lower bound is 16. The upper bound of *bsize* is 64 bytes and the lower bound is 8 bytes. The minimum value of *bsize* is also the lowest bound supported by the simulator. The upper bound of *assoc* is 16 and the lower bound is 1. There are 120 combinations of three parameters which is much larger than the number of combinations in Lin et al.’s work [7]. According to the statistic of more than forty benchmarks, there is no any value of the three parameters abandoned by every benchmark. The extension of searching space is necessary because the instability of a phase is larger than that of entire execution of a program.

2.2 Architecture of Reconfigurable Cache

The number of blocks in a set is limited to be powers of two to enable simple decoding. The blocks are physically divided into 6 sub arrays: 256, 128, 64, 32, 16, and 16 blocks. The partition makes it possible to tune from 512 to 16 *nblks*. The unused blocks are shutdown for saving energy

consumption.

An approach introduced by Chen et al. [3] can adjust the line size by configuring a counter which indicates the number of words to be read from the off-chip memory when there is a miss. The associativity tuning is implemented by Banerjee et al. [1]. A simple hardware counter keeps track of the miss rate and the information is fed back to the way controller. Each cache way can be selectively enable or disable by the controller.

In an non-tuned cache, the size of tag adjusts to $32 - \log_2(nblks \times bsize)$ under 32bits addressing mode. Dynamically tuning cache size during an execution of programs changes tag size as well. Even so, implementing a multi-length tag comparator is not considered efficient in this work. A feasible solution is the overlapped wide-tag [3] which extends the size of tag to maximum length in search space. Although some bits of tag may be redundant, it makes cache be any size without additional control hardware.

2.3 Energy Calculation

With the increasing of working frequency, the dynamic power consumption dominates most energy consumption in caches, so the static power consumption is not considered. The energy consumption caused by off-chip memory should not be ignored since the accessing of off-chip memory is a part of a normal memory access. When a cache misses, the instruction fetching and data referencing cause huge miss penalty. Besides the cache inner calculates, the off-chip accessing is included.

To calculate the energy consumed in the proposed cache architecture, the adopted energy model should be calculated fast and easily. We modify the energy model used by Shiue and Chakrabarti [10] for calculating the energy consumption to support cache reconfiguration. This model shows in (1) that is separated by four components.

E_{dec} is the energy consumption of the decoding logic and translating in address buses. The capacitance of a decoding logic is usually much less than that of an address bus; hence the energy consumption of the address buses dominates the entire address decoding path. E_{cell} denotes the energy consumption of the read/write circuitry and cell array due to status bit, tag, and data cell arrays. State bit, tag cell, and data cell consume most energy in cell array, and they can be implemented in dynamic or static logic. In dynamic circuit design, word/bit lines are pre-charged on every access. The energy consumed on bit lines is the same whether the values on the bit lines are 1 or 0.

In a static circuit design, there is no pre-charge on word/bit lines. The energy consumption on tag and data memory array directly depends on the bit switch of the word/bit lines. For above reason we only compute the dynamic energy consumption. It will access off-chip memory via address/data pad when cache misses.

E_{io} represents the energy consumption in I/O pad. In current microprocessors, the capacitance of I/O pads is usually larger than that on address and data buses. Therefore the energy consumption of I/O pad is dominated by the energy consumed during the bit switch of the I/O path. E_{mem} stands for accessing an off-chip memory. The energy model adopted in this work is stated as follows:

$$\begin{aligned} E_{total} &= E_{dec} + E_{cell} + E_{io} + E_{main} \quad (1) \\ E_{dec} &= \alpha \times Pr_A \times W_{add} \times (C/L) \times (N_{hit} + N_{miss}) \\ E_{cell} &= \beta \times [W \times (8 \times L + T + st)] \times [C / (W \times L) \\ &\quad + 4.8] \times (N_{hit} + N_{miss} - N_{WPhits}) + \beta \times (8 \times \\ &\quad L + T + St) \times [C / (W \times L) + 4.8] \times N_{WPhits} \\ E_{io} &= \gamma \times (Pr_D \times 8 \times L + Pr_A \times W_{add}) \times N_{miss} \\ E_{mem} &= \gamma \times (Pr_D \times 8 \times L + Pr_A \times W_{add}) \times N_{miss} + \\ &\quad E_m \times 8 \times L \times N_{miss} \end{aligned}$$

where

Pr_A = the probability of one bit switch in address bus/pad

Pr_D = the probability of one bit switch in data pad

W_{add} = the width of address bus

C = cache size in bytes

L = cache line size in bytes

W = way associativity

N_{hit} = number of hits

N_{miss} = number of miss

N_{WPhits} = number of way prediction hits

T = tag size in bits

St = number of status bits per block frame

E_m = energy consumption of a main memory access

$\alpha = 7.89e-17$; $\beta = 1.44e-14$; $\gamma = 5.45e-11$

α, β, γ are used for 0.8 μm CMOS technology

Su and Despain [11] revealed the relationship between bit switch rate and miss rate. An average bit switch rate is chosen as 4K bytes for cache size, 16 Bytes for cache line size, and 2-way for way associativity. By Su et al. [12], the value of Pr_A is 0.0045, and Pr_D is 0.0044. Clearly, it takes at least 14 bits address bus to access 16 Kbytes cache size. The SRAM CY7C1326-133 from Cypress is used as our cache memory. The size of the SRAM is 2Mbits, access time 4ns, voltage of 3.3V, energy consumption of 4.95nJ per access. St is the symbol of a valid bit which is the only one status bit in the proposed scheme.

3 Real-time Reconfigurable Cache

3.1 Algorithm

The algorithm used in this work is based on Lin et al.'s work [7] and the algorithm is modified to fit the real-time scheme. The algorithm tunes one of three parameters at a time. When the low point is reached, the algorithm goes to the next parameter. The proposed algorithm has three input sets: nblks, bsize, and assoc. In each interval, number of cache misses can be generated by miss counter and sent to the energy calculator. The following is the general model of the algorithm.

```

run the application in the time interval
while( time is up )
{
    current energy = energy calculator( nblks, bsize,
    assoc, misses, length of interval );
    if( optimal energy > current energy )
        optimal value = current value;

    if( optimal value is minimum or maximum in
    search space )
        if( value next to the optimal value is checked )
            optimal value of the parameter is found;
        else
            next value = the value next to the optimal
            value;
    else
        if( both sides of optimal value is checked )
            optimal value of the parameter is found;
        else
            if( value = 1/2 × optimal value is checked )
                next value = 2 × optimal value;
            else
                next value = 1/2 × optimal value;

    if( optimal value of the parameter is found )
        goto next parameter;
    else
        current value = next value;
}

```

The proposed algorithm searches for optimal value of each parameter sequentially. As the optimal energy and current energy are the only information available, the direction of tuning is unknown when the algorithm begins. For the consistency of implementation, a smaller value is always chosen to be the next configuration at first. If a value is selected as the optimal parameter, both larger and smaller ones around the optimal parameter must have been tested. If the value is the minimum or maximum in the search space, only one side has to be tested.

3.2 The Priority of Three Parameters

The algorithm introduced in above section tunes one of three parameters at a time. The priority of parameter tuning is decided by the order of impact on energy. Figure 1 is the analysis of the energy model stated in Section 2.5. The x -axis is the number of misses during 1 million instructions and the y -axis is the average percentage of energy increasing when one parameter doubles its value.

As shown in Figure 1, when the number of misses is less than 100, the assoc is the most influential parameter. When the number of misses is larger than 100, the impact of assoc and nblks decays in a fast speed while the impact of bsize increases on the contrary. Based on statistics of the simulation, the number of misses under most cache configuration is greater than 100 in 1 million instructions. The priority of parameters is decided as bsize, assoc and nblks.

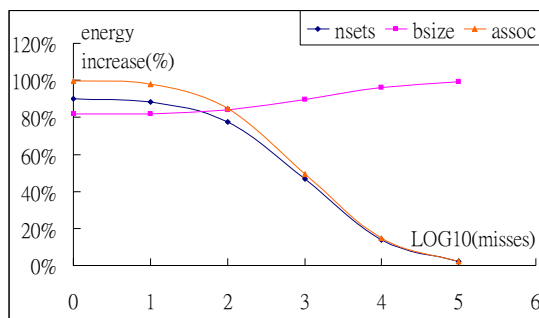


Figure 1. Analysis of the energy model.

3.3 Statistic Starting Point

An important observation has been introduced in Lin et al.'s work [7]. With the increasing of nblks, the arc of energy consumption versus nblks should have only one low point for any benchmark and so are the other two parameters. The red line in Figure 2 is not available in the simulation of this work. The phenomenon is apparent when one parameter is tuned. The situation gets much complicated if there are three parameters tuning simultaneously.

Starting point is the initial value of cache configuration when the proposed algorithm begins. If the starting point is too far from the low point, it is possible for the cache configuration tuning to be trapped in sub-low points. To avoid this case, the starting point should be near the low point. After gathering statistics of all benchmarks in this work, the values of each parameter appear most in optimal configurations are chosen as starting point.

The nblks is set as 64, bsize as 16 bytes and assoc as 8 to the configuration of starting point for L1 instruction cache.

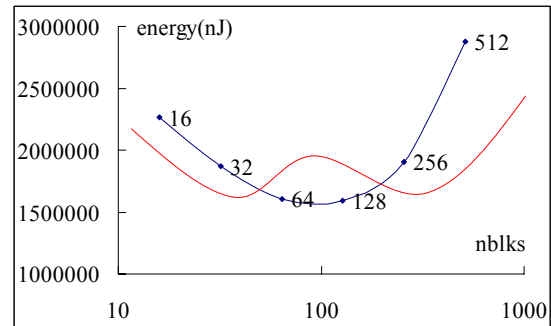


Figure 2. The energy consumption of nblks from 16 to 512. The benchmark is bitcount, bsize is 8 bytes and assoc is 4. The red line is a synthetic example which is not available in this work.

3.4 Analysis of average Tuning Time (table)

According to the algorithm in section 3.1, the best case of tuning time is 1 + 1 + 1 and the worst case of tuning time is 6 + 5 + 4. The best case and the worst case are the same as Lin et al.'s work [7]. The average tuning time of nblks is given in Table 1.

Table 1. Tuning Time of All Cases for Current nblks to Next nblks

$\begin{matrix} Y \\ X \end{matrix}$	512	256	128	64	32	16	sum
512	2	3	4	5	6	5	25
256	2	3	3	4	5	4	21
128	3	4	3	3	4	3	20
64	4	5	4	3	3	2	21
32	5	6	5	4	3	1	24
16	5	6	5	4	3	2	25
sum							136

Each element in Table 1 records the steps needed when tuning from any nblks in column X to next nblks in row Y . For example, when current nblks is 512 and the next optimal nblks is 32, it takes 4 steps to go from 512 to 32. There is no information whether the value 16 is better or not, so it is necessary to change nblks to 16. After applying try and error strategy, the value 16 is found worse than 32 and the nblks is changed back to 32. It totally costs 6 tune times as shown in Table 1. The assumption that the probability of next nblks is average to every nblks value is adopted in this analysis.

The possibility of next nblks is 1/6 to each

value so the average tuning time from nblks 512 to next nblks is 25/6. Because the possibility of current nblks is also 1/6 for each nblks, the average tuning time for nblks is 136/36. By the same estimation, the average tuning time of bsize is 45/16 and the average tuning time of assoc is 83/25. The overall average tuning time of the proposed algorithm is the sum of average cases of three parameters which equals 9.91. When the proposed algorithm is applied, it tunes 9.91 times on average to find an optimal configuration.

3.5 Real-time Reconfigurable Scheme

The idea of real-time reconfiguration is dividing the execution of an application into constant time intervals. The interval between tuning is set to be 1 million instructions in this work. During each interval, the proposed algorithm is applied to find the optimal cache configuration and the found configuration is recorded for the next round. The optimal configuration found in an interval is called a local optimal configuration while the optimal cache configuration found during the whole execution is called global optimal one. A local optimal configuration may not be optimal for the next interval because the behavior of the application may differ. A reasonable scheme is shifting the recorded local optimal configuration backward to the start of the interval when the second round is coming. This scheme ensures that the applied optimal configuration is suitable for the current time interval and saves energy in a significant degree.

4 Experimental Methodology

4.1 Environment

SimpleScalar introduced by Buger et al. [2], a MIPS-like microprocessor simulator, is the tool set used in this work. This tool is developed by University of Wisconsin, Madison. We carefully modify and verify the SimpleScalar tool set to support real-time cache reconfiguration. Mibench benchmark suit is taken for embedded systems simulation that is developed by University of Michigan, Ann Arbor [5]. The simulation is under the cache which is modeled as in-order execution, write-back and the replacement policy is LRU.

4.2 Real-time Reconfiguration Results

The proposed reconfigurable scheme saves energy by finding local optimal configuration

which is considered better than global optimal one. The dijkstra benchmark calculates the shortest path between every pair of nodes in a graph. The global optimal cache configuration of dijkstra is 64 for nblks, 8 bytes for bsize and 16 for assoc. As shown in Figure 3, the global optimal configuration is not always the best configuration in the execution. There is another configuration better than the global one periodically. The purpose of the proposed scheme is always to select the cache configuration with lowest energy consumption.

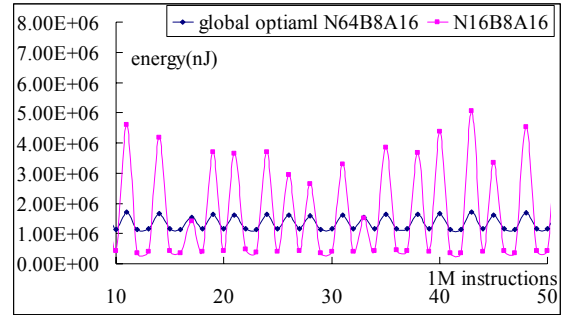


Figure 3. The energy consumption of dijkstra benchmark with different cache configurations.

However, not every benchmark in Mibench is sensible to cache reconfiguration. The sha benchmark in Figure 4 is a secure hash algorithm that produces a 160-bit message digests for a given input. The global optimal configuration of sha dominates the whole execution and there is no configuration better in any time interval. To these insensible benchmarks, the proposed scheme still makes sense. Although it is in vain for the proposed scheme to tune cache configuration, the global optimal cache configuration of insensible applications will be found by the scheme and the insensibility will be revealed.

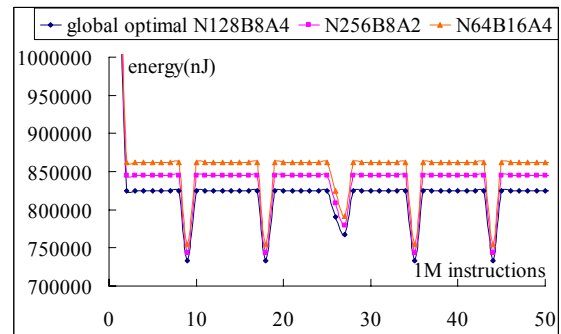


Figure 4. The energy consumption of sha benchmark with different cache configurations.

Figure 5 shows the overall energy saving on Mibench. The proposed scheme achieves on

average 7.59% energy reduction compared to respective global optimal cache configurations of all benchmarks. Besides some cache tuning insensible benchmarks, benchmarks with tiny number of instructions benefit nothing from the proposed scheme because the time interval is even larger.

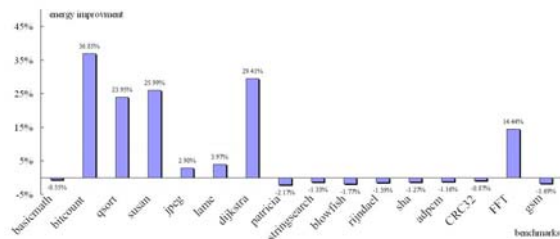


Figure 5. The average energy reduction for each benchmark of Mibench.

5 Conclusions

Saving energy is undoubtedly an important issue for embedded systems. As embedded systems work in a quite simple environment, it makes sense to optimize cache configuration to a certain application. Lin et al. proposed an on-line reconfigurable cache architecture which reduces energy over a non-tuned reference cache [7]. This work improves on average 7.59% energy reduction over Lin and Duh's work and achieves more than 60% energy saving compared to traditional four-way set-associativity and directed-mapped caches. If the target application is insensible to cache tuning, the global optimal cache configurable is found and the insensibility is shown by the proposed scheme.

References

- [1] S. Banerjee, Surendra G and S. K. Nandy. Program phase directed dynamic cache way reconfiguration for power efficiency. *Design Automation Conference, Asia and South Pacific*, pp. 884–889, 2007.
- [2] D. Buger, and T. M. Austin. The SimpleScalar tool set, version 2.0. *ACM SIGARCH Computer Architecture News*, Dallas, TX, USA, ACM, pp. 13–25, 1997.
- [3] L.-M. Chen, X.-C. Zou, J.-M. Lei, and Z.-L. Liu. Dynamically reconfigurable cache for low-power embedded system. *Third International Conference on Natural Computation*, vol. 5, pp. 180–184, 2007.
- [4] A. Gordon-Ross, P. Viana, F. Vahid, W. Najjar, and E. Barros. A one-shot configurable-cache tuner for improved energy and performance. *Design, Automation, and Test in Europe*, pp. 1–6, 2007.
- [5] M. R. Guthaus, J. S. Ringenberg, D. Ernst, T. M. Austin, T. Mudge, and R. B. Brown. MiBench: A free, commercially representative embedded benchmark suite. In *IEEE International Workshop on Workload Characterization*, Austin, TX, pp. 3–14, 2001.
- [6] J. Kin, M. Gupta, and W. Mangione-Smith. The Filter Cache: An Energy Efficient Memory Structure. In *Proceedings of the ACM/IEEE 30th Annual International Symposium on Microarchitecture*, Research Triangle Park, North CA, USA, pp. 184–193, 1997.
- [7] C.-H. Lin, and D.-R. Duh. On-line reconfigurable cCache for embedded systems. In *Proceedings of the 2006 Conf. on Information Technology and Applications in Outlying Islands*, Jinning, Kinmen, Taiwan, June 2–3, Taiwan, 2006.
- [8] S. Segars. Low power design techniques for microprocessors. *IEEE International Solid-State Circuits Conference Tutorial*, San Francisco, CA, USA, 2001.
- [9] T. Sherwood, S. Sair, and B. Calder. Phase tracking and prediction. In *Proceeding of the 30th Annual International Symposium on Computer Architecture*, pp. 336–347, 2003.
- [10] W.-T. Shiue and C. Chakrabarti. Memory design and exploration for low power, embedded systems. *Journal of VLSI Signal Processing-Systems for Signal, Image, and Video Technology*, vol. 29, no. 3, pp. 167–178, 2001.
- [11] C.-L. Su and A. Despain. Cache design trade-offs for power and performance optimization: A case study. In *Proceedings of the 1995 International Symposium on Low Power Design*, Dana Point, CA, USA, pp. 63–68, 1995.
- [12] C.-L. Su and A. M. Despain. Cache designs for energy efficiency. In *Proceedings of the 28th Hawaii International Conference on System Sciences*, Kihei, Maui, Hawaii, pp. 306–315, 1995.
- [13] C.-L. Yang and C.-H. Lee. HotSpot cache: Joint temporal and spatial locality exploitation for I-cache energy reduction. In *Proceeding of the International Symposium on Low Power Electronics and Design*, pp. 114–119, 2004.
- [14] C. Zhang, F. Vahid, and W. Najjar. A highly configurable cache for low energy embedded systems. *ACM Transactions Embedded Computing Systems*, vol. 4, no. 2, pp. 363–387, 2005.
- [15] C. Zhang, F. Vahid, and W. Najjar. Energy benefits of a configurable line size cache for

embedded systems. *In Proceedings of IEEE
Computer Society Annual Symposium on Very*

*Large Scale Integrtrion, Darmstadt, Germany,
pp. 87-91, 2003.*